Rupeeflow — Scale to 3 Crore Transactions/Day

Senior Developer Playbook: Database · Redis · Code Patterns

Date: March 2026 **Target:** 30,000,000 transactions/day **Stack:** NestJS · PostgreSQL · Redis · Bull · PM2
Classification: Internal Engineering — Confidential

Part 0 — The Math First

> ***Never optimize without knowing your numbers.***

```
3 Crore transactions/day = 30,000,000 / day

Average TPS   = 30,000,000 ÷ 86,400 seconds   = ~347 TPS
Peak TPS      = 347 × 3 (peak multiplier)       = ~1,050 TPS
1 payin flow  = 4 DB ops + 1 Redis read + 1 queue write
Peak DB ops   = 1,050 × 4                        = ~4,200 ops/sec
Peak Redis    = 1,050 × 1 = ~1,050 ops/sec
Queue depth   = assume 2s processing lag         = ~2,100 jobs in queue at peak

WHERE YOU ARE NOW (from db.config.ts comments):
  - Target: 200 req/sec
  - Actual: 45 req/sec, 64% success
  - Root cause: connection pool exhausted, no PgBouncer, no read replicas
```

Gap to bridge: 45 req/sec → 1,050 req/sec. That's 23x improvement needed.

You need ALL of the following — skipping any one will create a bottleneck:

Layer	Current State	Required State
DB Connections	TypeORM direct, no pool config	PgBouncer + pool tuned
DB Read	Single node (writes + reads)	1 write + 2 read replicas
DB Schema	No partitioning	Monthly partitioned payin_orders + transactions
Redis	Single node, no cluster	Redis Cluster (3 master + 3 replica)
Queue	Bull + shared queues	BullMQ + per-merchant priority lanes
App	6 PM2 instances	6–20 horizontal pods (K8s ready)
Async	sync payin DB insert via queue	Retained + race condition fixed

---

Part 1 — PostgreSQL: The Right Way

1.1 Connection Pooling with PgBouncer

The Problem: TypeORM opens up to `max` direct connections per Node process. With 6 PM2 instances × `max: 20` = 120 connections. PostgreSQL default `max_connections = 100`. You're already over the limit. That's why you see 64% success.

The Solution: PgBouncer in Transaction Mode

```
App (6 instances) → PgBouncer → PostgreSQL
     ↓                  ↓              ↓
 up to 500          1 process      max_connections
 app connections    proxies all    = 100 actual DB
                    to 80 real     connections
                    DB connections
```

Install PgBouncer (Docker Compose):

```yaml
# docker-compose.yml — add this service
pgbouncer:
  image: bitnami/pgbouncer:1.22.1   # pin the version
  environment:
    POSTGRESQL_HOST: postgres
    POSTGRESQL_PORT: 5432
    POSTGRESQL_USERNAME: ${DB_USERNAME}
    POSTGRESQL_PASSWORD: ${DB_PASSWORD}
    POSTGRESQL_DATABASE: ${DB_NAME}
    PGBOUNCER_POOL_MODE: transaction     # CRITICAL: transaction mode
    PGBOUNCER_MAX_CLIENT_CONN: 1000      # total app connections allowed
    PGBOUNCER_DEFAULT_POOL_SIZE: 80      # actual DB connections PgBouncer holds
    PGBOUNCER_MIN_POOL_SIZE: 20
    PGBOUNCER_RESERVE_POOL_SIZE: 10
    PGBOUNCER_RESERVE_POOL_TIMEOUT: 5
    PGBOUNCER_SERVER_IDLE_TIMEOUT: 600
    PGBOUNCER_CLIENT_IDLE_TIMEOUT: 60
    PGBOUNCER_AUTH_TYPE: scram-sha-256
  ports:
    - "6432:6432"
  depends_on:
    - postgres
```

TypeORM config after PgBouncer (`src/config/db.config.ts`):

```typescript
export const dbConfig: TypeOrmModuleOptions = {
  type: "postgres",
  host: process.env.PGBOUNCER_HOST || host,   // point to PgBouncer, not Postgres
  port: 6432,                                  // PgBouncer port
  username,
  password,
  database: name,
  entities: [__dirname + "/../**/*.entity{.ts,.js}"],
  synchronize: false,
  maxQueryExecutionTime: 2000,
  ssl: false,
```

```
  extra: {
    // With PgBouncer transaction mode:
    // - Each request gets a connection from PgBouncer
    // - Connection returned immediately after query
    // - Prepared statements MUST be disabled (prepareThreshold: 0) — already done ✓
    max: 25,                          // per Node process → 6 × 25 = 150
app→PgBouncer
    min: 5,
    connectionTimeoutMillis: 10000,
    idleTimeoutMillis: 30000,
    statement_timeout: 8000,
    lock_timeout: 3000,
    idle_in_transaction_session_timeout: 5000,
    prepareThreshold: 0,              // MANDATORY with PgBouncer transaction mode ✓
  },
};
```

IMPORTANT: With PgBouncer transaction mode, you **cannot use** `SET` commands, `LISTEN/NOTIFY`, advisory locks, or session-level prepared statements. All of these are session-scoped. If you use `QueryRunner` for transactions (which you do in `PayinProcessor`), each `BEGIN/COMMIT` block gets the same connection — this is safe.

1.2 Read Replicas — Separate OLTP from OLAP

The Problem: Your analytics queries (`business-trend` , `hourly-analytics` , `conversion-rate`) are running `GROUP BY` , `COUNT(*)` , date-range scans on the same DB as your live payment inserts. At 1,050 TPS, a 5-second analytics query is holding shared buffers and creating lock contention.

The Solution: Two Connections in NestJS

```
// src/config/db.config.ts — add a read replica config

export const dbReadReplicaConfig: TypeOrmModuleOptions = {
  type: "postgres",
  replication: {
    master: {
      host: process.env.DB_WRITE_HOST,      // primary (writes)
      port: 6432,
      username,
      password,
      database: name,
    },
    slaves: [
      {
        host: process.env.DB_READ_HOST_1,   // replica 1 (reads)
        port: 6432,
        username,
        password,
        database: name,
      },
      {
```

```
        host: process.env.DB_READ_HOST_2,   // replica 2 (analytics only)
        port: 6432,
        username,
        password,
        database: name,
      },
    ],
  },
  entities: [__dirname + "/../**/*.entity{.ts,.js}"],
  synchronize: false,
  extra: {
    max: 20,
    min: 3,
    prepareThreshold: 0,
  },
};
```

TypeORM automatically routes `find()`, `findOne()`, `createQueryBuilder()` SELECT queries to slaves and all writes to master. Zero code changes in repositories.

For analytics — use the second replica explicitly:

```
// src/modules/analytics/analytics.repository.ts
import { DataSource } from "typeorm";
import { InjectDataSource } from "@nestjs/typeorm";

@Injectable()
export class AnalyticsRepository {
  constructor(
    @InjectDataSource("ANALYTICS_DB")   // named connection to replica 2
    private readonly analyticsDb: DataSource,
  ) {}

  async getBusinessTrend(userId: string, start: Date, end: Date) {
    // This query NEVER touches your write DB
    return this.analyticsDb.query(`
      SELECT
        DATE_TRUNC('day', created_at) as date,
        COUNT(*) FILTER (WHERE status = 'SUCCESS') as success_count,
        SUM(amount) FILTER (WHERE status = 'SUCCESS') as success_volume,
        COUNT(*) FILTER (WHERE status = 'FAILED') as failed_count
      FROM payin_orders_2026_03   -- partitioned table (see 1.3)
      WHERE user_id = $1
        AND created_at BETWEEN $2 AND $3
      GROUP BY 1
      ORDER BY 1
    `, [userId, start, end]);
  }
}
```

1.3 Table Partitioning — The Biggest Performance Win

The Problem: `payin_orders` and `transactions` will have 30M+ rows/month. A single unpartitioned table with a B-tree index on `created_at` is ~15 GB/month. Query planners stop using indexes efficiently past ~50M rows. Analytics scans become full-table.

The Solution: Range Partitioning by Month

```sql
-- Migration: partition payin_orders by created_at (monthly)

-- Step 1: Rename existing table
ALTER TABLE payin_orders RENAME TO payin_orders_old;

-- Step 2: Create partitioned parent table
CREATE TABLE payin_orders (
  id            BIGSERIAL,
  order_id      VARCHAR(30) NOT NULL,
  user_id       VARCHAR(30) NOT NULL,
  amount        DECIMAL(15, 2) NOT NULL,
  status        VARCHAR(20) NOT NULL DEFAULT 'INITIATED',
  payment_method VARCHAR(30),
  utr           VARCHAR(100),
  commission_id  VARCHAR(30),
  settlement_status VARCHAR(20) DEFAULT 'UNSETTLED',
  checkout_data  JSONB,
  created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  -- Add all your existing columns here
  PRIMARY KEY (id, created_at)   -- partition key MUST be in PK
) PARTITION BY RANGE (created_at);

-- Step 3: Create monthly partitions
CREATE TABLE payin_orders_2026_01 PARTITION OF payin_orders
  FOR VALUES FROM ('2026-01-01') TO ('2026-02-01');

CREATE TABLE payin_orders_2026_02 PARTITION OF payin_orders
  FOR VALUES FROM ('2026-02-01') TO ('2026-03-01');

CREATE TABLE payin_orders_2026_03 PARTITION OF payin_orders
  FOR VALUES FROM ('2026-03-01') TO ('2026-04-01');

-- Step 4: Indexes on each partition (PostgreSQL 11+ propagates from parent)
CREATE INDEX idx_payin_orders_user_created ON payin_orders (user_id, created_at
DESC);
CREATE INDEX idx_payin_orders_order_id ON payin_orders (order_id);
CREATE INDEX idx_payin_orders_status_created ON payin_orders (status, created_at
DESC);

-- Step 5: Unique constraint on order_id across all partitions
CREATE UNIQUE INDEX idx_payin_orders_order_id_unique ON payin_orders (order_id,
created_at);
```

Auto-create next month's partition — add this as a cron job:

```
// src/modules/maintenance/partition-manager.service.ts
@Injectable()
export class PartitionManagerService {
  constructor(
    @InjectDataSource() private dataSource: DataSource,
  ) {}

  // Run via @Cron('0 0 25 * *') — 25th of every month
  @Cron("0 0 25 * *")
  async createNextMonthPartition() {
    const next = new Date();
    next.setMonth(next.getMonth() + 1);

    const partitionName =
`payin_orders_${next.getFullYear()}_${String(next.getMonth() + 1).padStart(2,
"0")}`;
    const startDate = new Date(next.getFullYear(), next.getMonth(), 1);
    const endDate = new Date(next.getFullYear(), next.getMonth() + 1, 1);

    await this.dataSource.query(`
      CREATE TABLE IF NOT EXISTS ${partitionName}
        PARTITION OF payin_orders
        FOR VALUES FROM ('${startDate.toISOString()}') TO
('${endDate.toISOString()}')
    `);

    // Same for transactions table
    const txPartitionName =
`transactions_${next.getFullYear()}_${String(next.getMonth() + 1).padStart(2,
"0")}`;
    await this.dataSource.query(`
      CREATE TABLE IF NOT EXISTS ${txPartitionName}
        PARTITION OF transactions
        FOR VALUES FROM ('${startDate.toISOString()}') TO
('${endDate.toISOString()}')
    `);
  }
}
```

Query performance impact:

- Before partitioning: `SELECT WHERE created_at > '2026-03-01'` → scans 200M rows
- After partitioning: same query → scans only `payin_orders_2026_03` (~30M rows, 85% smaller)
- PostgreSQL uses **partition pruning** — it never touches other month's tables

---

1.4 Indexing Strategy — Only What You Need

Indexes you must have (add via migration):

```sql
-- payin_orders: the 5 most common query patterns
CREATE INDEX CONCURRENTLY idx_po_user_status_created
  ON payin_orders (user_id, status, created_at DESC);

CREATE INDEX CONCURRENTLY idx_po_settlement_status
  ON payin_orders (settlement_status, user_id)
  WHERE settlement_status = 'UNSETTLED';    -- partial index — huge win

CREATE INDEX CONCURRENTLY idx_po_order_id
  ON payin_orders (order_id);               -- webhook lookup

-- transactions: analytics
CREATE INDEX CONCURRENTLY idx_tx_user_type_created
  ON transactions (user_id, type, created_at DESC);

-- wallet: wallet update by userId (must be fast for every webhook)
CREATE INDEX CONCURRENTLY idx_wallet_user_id
  ON wallets (user_id);

-- user_integration_mapping: routing lookup (called on every payin)
CREATE INDEX CONCURRENTLY idx_uim_user_id
  ON user_integration_mapping (user_id);
```

Indexes that are KILLING your performance (check and drop unused ones):

```sql
-- Find unused indexes (run this monthly)
SELECT
  schemaname,
  tablename,
  indexname,
  idx_scan,
  pg_size_pretty(pg_relation_size(indexrelid)) as index_size
FROM pg_stat_user_indexes
WHERE idx_scan < 100
  AND indexrelid NOT IN (
    SELECT conindid FROM pg_constraint WHERE contype IN ('p','u')
  )
ORDER BY pg_relation_size(indexrelid) DESC;
```

1.5 The Wallet Update Pattern — Fix the Critical Race Condition

Your current pattern: TypeORM optimistic locking (`version` column) + no retry = silent failures at high concurrency.

The Right Pattern: PostgreSQL Advisory Lock + Explicit Retry

```typescript
// src/modules/wallet/wallet.service.ts

@Injectable()
export class WalletService {
```

```
constructor(
  @InjectRepository(WalletEntity)
  private readonly walletRepo: Repository<WalletEntity>,
  @InjectDataSource()
  private readonly dataSource: DataSource,
) {}

/**
 * Atomically credit wallet after successful payin webhook.
 * Uses PostgreSQL advisory lock to serialize concurrent updates
 * for the same user. No optimistic lock retries needed.
 */
async creditWalletForPayin(
  userId: string,
  netAmount: number,
  commissionAmount: number,
  queryRunner?: QueryRunner,
): Promise<void> {
  const runner = queryRunner ?? this.dataSource.createQueryRunner();
  const isOwnRunner = !queryRunner;

  if (isOwnRunner) {
    await runner.connect();
    await runner.startTransaction("READ COMMITTED");
  }

  try {
    // Advisory lock: serializes all wallet updates for this user
    // hashtext() converts userId string to a stable integer
    await runner.query(
      `SELECT pg_advisory_xact_lock(hashtext($1))`,
      [userId],
    );

    // Now this is safe — only one transaction holds the lock
    await runner.query(
      `UPDATE wallets
       SET
         total_collections = total_collections + $2,
         total_payin_balance = total_payin_balance + $2,
         available_payout_balance = available_payout_balance + $3,
         updated_at = NOW()
       WHERE user_id = $1`,
      [userId, netAmount + commissionAmount, netAmount],
    );

    if (isOwnRunner) await runner.commitTransaction();
  } catch (err) {
    if (isOwnRunner) await runner.rollbackTransaction();
    throw err;
  } finally {
    if (isOwnRunner) await runner.release();
```

```
    }
  }

  /**
   * Debit wallet for payout — checks balance atomically.
   * Returns false if insufficient balance (caller retries or fails gracefully).
   */
  async debitWalletForPayout(
    userId: string,
    amount: number,
    queryRunner: QueryRunner,
  ): Promise<boolean> {
    await queryRunner.query(
      `SELECT pg_advisory_xact_lock(hashtext($1))`,
      [userId],
    );

    const result = await queryRunner.query(
      `UPDATE wallets
       SET
         available_payout_balance = available_payout_balance - $2,
         updated_at = NOW()
       WHERE user_id = $1
         AND available_payout_balance >= $2    -- atomically check balance
       RETURNING id`,
      [userId, amount],
    );

    return result.length > 0;  // false = insufficient balance
  }
}
```

Why this is better than optimistic locking:

- No `OptimisticLockVersionMismatchError` exceptions
- No retry loops needed
- PostgreSQL advisory lock is released automatically when transaction ends
- Works correctly with PgBouncer transaction mode

1.6 Slow Query Detection (Add to Production)

```
// src/config/db.config.ts — add this listener
export const dbConfig: TypeOrmModuleOptions = {
  // ...existing config...

  // Log slow queries in production
  maxQueryExecutionTime: 500,  // log anything > 500ms
  logger: "advanced-console",

  // Custom slow query hook — send to your alerting system
  subscribers: [],  // add TypeORM subscriber for query events
```

```
};

// In your AppModule or a health module:
// Run this SQL on startup to enable pg_stat_statements (needs superuser once)
// CREATE EXTENSION IF NOT EXISTS pg_stat_statements;
//
// Then query it to find your top 10 slowest queries:
// SELECT query, calls, mean_exec_time, total_exec_time
// FROM pg_stat_statements
// ORDER BY mean_exec_time DESC LIMIT 10;
```

Part 2 — Redis: The Right Way

2.1 Redis Cluster — Eliminate the SPOF

Current: Single Redis → crash = no auth, no queues, no 2FA. **Required:** Redis Cluster with 3 masters + 3 replicas.

```yaml
# docker-compose.yml — Redis Cluster (dev/staging)
redis-node-1:
  image: redis:7.2.4    # pin version, never use :latest
  command: >
    redis-server
    --cluster-enabled yes
    --cluster-config-file nodes.conf
    --cluster-node-timeout 5000
    --appendonly yes
    --maxmemory 2gb
    --maxmemory-policy allkeys-lru
    --save 900 1
    --save 300 10
  ports: ["6381:6379"]
  volumes: ["redis-data-1:/data"]

redis-node-2:
  image: redis:7.2.4
  command: >
    redis-server
    --cluster-enabled yes
    --cluster-config-file nodes.conf
    --cluster-node-timeout 5000
    --appendonly yes
    --maxmemory 2gb
    --maxmemory-policy allkeys-lru
  ports: ["6382:6379"]
  volumes: ["redis-data-2:/data"]

redis-node-3:
  image: redis:7.2.4
  command: >
    redis-server
```

```
    --cluster-enabled yes
    --cluster-config-file nodes.conf
    --cluster-node-timeout 5000
    --appendonly yes
    --maxmemory 2gb
    --maxmemory-policy allkeys-lru
  ports: ["6383:6379"]
  volumes: ["redis-data-3:/data"]
```

NestJS Redis Cluster config:

```
// src/config/redis.config.ts
import { BullModuleOptions } from "@nestjs/bull";

export const redisClusterConfig = {
  // For ioredis (used by Bull/BullMQ and cache-manager)
  clusters: [
    { host: process.env.REDIS_NODE_1_HOST, port: 6379 },
    { host: process.env.REDIS_NODE_2_HOST, port: 6379 },
    { host: process.env.REDIS_NODE_3_HOST, port: 6379 },
  ],
  enableReadyCheck: true,
  redisOptions: {
    password: process.env.REDIS_PASSWORD,
    connectTimeout: 10000,
    maxRetriesPerRequest: 3,
    lazyConnect: false,
  },
  clusterRetryStrategy: (times: number) => {
    if (times > 10) return null;  // stop retrying after 10 attempts
    return Math.min(100 * times, 3000);
  },
};

// For BullMQ queues
export const bullMQRedisConfig: BullModuleOptions = {
  connection: {
    clusters: redisClusterConfig.clusters,
    ...redisClusterConfig.redisOptions,
    enableOfflineQueue: false,    // don't buffer jobs if Redis is down
    enableAutoPipelining: true,   // batch Redis commands automatically
  },
};
```

2.2 Key Design — The Non-Negotiable Rules

Rule 1: Always use a namespace prefix Rule 2: Always set a TTL — no key should live forever Rule 3: Use hashes for structured data — not JSON strings Rule 4: Use consistent naming: `{namespace}:{entity}:{id}:{field?}`

```
// src/constants/redis-cache.constant.ts — REWRITE this file

export const REDIS_KEYS = {
  // Auth
  USER: (userId: string) => `user:${userId}`,                          // TTL: 24h
  USER_ROLE: (userId: string) => `user:${userId}:role`,               // TTL: 1h
  API_KEY: (clientId: string) => `apikey:${clientId}`,                // TTL: 30d
  API_KEY_WHITELIST: (clientId: string) => `apikey:${clientId}:ips`,  // TTL: 1h
  TWO_FA_OTP: (userId: string) => `2fa:otp:${userId}`,               // TTL: 5m
  TWO_FA_PENDING: (userId: string) => `2fa:pending:${userId}`,       // TTL: 5m
  LOGIN_ATTEMPTS: (email: string) => `login:attempts:${email}`,      // TTL: 30m
  LOGIN_LOCK: (email: string) => `login:lock:${email}`,              // TTL: 30m

  // OTPs
  SIGNUP_OTP: (mobile: string) => `otp:signup:${mobile}`,            // TTL: 15m
  FORGOT_PWD_OTP: (mobile: string) => `otp:forgot:${mobile}`,       // TTL: 15m

  // Integration routing (called on EVERY payin — must be fast)
  USER_INTEGRATION: (userId: string) => `integration:user:${userId}`,// TTL: 1h

  // Commission plans (invalidate on change)
  COMMISSION_PLAN: (userId: string, type: string) => `commission:${userId}:${type}`,
// TTL: 1h

  // Stats (time-bounded cache)
  STATS_MERCHANT: (userId: string, date: string) => `stats:m:${userId}:${date}`,
// TTL: 10m
  STATS_ADMIN: (date: string) => `stats:a:${date}`,
// TTL: 5m
  STATS_ADMIN_WEEK: (weekStart: string) => `stats:a:week:${weekStart}`,
// TTL: 1h

  // Idempotency keys (CRITICAL — prevents duplicate payins)
  PAYIN_IDEMPOTENCY: (key: string) => `idem:payin:${key}`,           // TTL: 24h
  WEBHOOK_IDEMPOTENCY: (orderId: string, pgRef: string) =>
    `idem:webhook:${orderId}:${pgRef}`,                              // TTL: 48h

  // Distributed locks
  WALLET_LOCK: (userId: string) => `lock:wallet:${userId}`,          // TTL: 10s
  SETTLEMENT_LOCK: (settlementId: string) => `lock:settlement:${settlementId}`, //
TTL: 30s

  // Payment status (webhook result polling)
  PAYMENT_STATUS: (orderId: string) => `payment:status:${orderId}`,  // TTL: 48h

  // Ertech token
  ERTECH_TOKEN: () => `ext:ertech:token`,                            // TTL: 55m
(refresh before 1h)
} as const;

// TTL constants in seconds
```

```
export const REDIS_TTL = {
  USER: 86400,           // 24h
  USER_ROLE: 3600,       // 1h
  API_KEY: 2592000,      // 30d
  API_KEY_WHITELIST: 3600,
  TWO_FA: 300,           // 5m
  OTP: 900,              // 15m
  INTEGRATION: 3600,     // 1h
  COMMISSION: 3600,      // 1h
  STATS_REALTIME: 60,    // 1m — for dashboards
  STATS_DAILY: 600,      // 10m
  STATS_WEEKLY: 3600,    // 1h
  IDEMPOTENCY_PAYIN: 86400,   // 24h
  IDEMPOTENCY_WEBHOOK: 172800, // 48h
  LOCK: 10,              // 10s — for distributed locks
  PAYMENT_STATUS: 172800, // 48h
} as const;
```

2.3 Cache Aside Pattern — The Standard Pattern for Your Use Case

```
// src/shared/services/cache.service.ts
// A typed wrapper around your Redis client

@Injectable()
export class CacheService {
  private readonly logger = new Logger(CacheService.name);

  constructor(
    @Inject(CACHE_MANAGER) private cacheManager: Cache,
  ) {}

  /**
   * Get from cache. If miss, call loader(), store result, return it.
   * This is the Cache-Aside pattern.
   */
  async getOrSet<T>(
    key: string,
    loader: () => Promise<T>,
    ttlSeconds: number,
  ): Promise<T> {
    const cached = await this.cacheManager.get<T>(key);
    if (cached !== undefined && cached !== null) {
      return cached;
    }

    const value = await loader();

    if (value !== null && value !== undefined) {
      await this.cacheManager.set(key, value, ttlSeconds * 1000);
    }
```

```
    return value;
  }

  /**
   * Invalidate one or more keys.
   * Call this whenever you update the underlying data.
   */
  async invalidate(...keys: string[]): Promise<void> {
    await Promise.all(keys.map(k => this.cacheManager.del(k)));
  }

  async set<T>(key: string, value: T, ttlSeconds: number): Promise<void> {
    await this.cacheManager.set(key, value, ttlSeconds * 1000);
  }

  async get<T>(key: string): Promise<T | null> {
    return this.cacheManager.get<T>(key) ?? null;
  }

  async exists(key: string): Promise<boolean> {
    const val = await this.cacheManager.get(key);
    return val !== null && val !== undefined;
  }

  async setNX(key: string, value: string, ttlSeconds: number): Promise<boolean> {
    // SET key value NX EX ttl — atomic set-if-not-exists
    // Used for distributed locks and idempotency
    const result = await (this.cacheManager.store as any).client.set(
      key, value, "EX", ttlSeconds, "NX"
    );
    return result === "OK";
  }

  async del(key: string): Promise<void> {
    await this.cacheManager.del(key);
  }
}
```

Usage in your AuthGuard (replacing the inline Redis calls):

```
// src/guard/auth.guard.ts — proper cache-aside usage
async canActivate(context: ExecutionContext): Promise<boolean> {
  // ...JWT verification...

  const user = await this.cacheService.getOrSet(
    REDIS_KEYS.USER(userId),
    () => this.usersRepository.findOne({
      where: { id: userId },
      select: ["id", "role", "accountStatus", "onboardingStatus", "email"],
    }),
```

```
    REDIS_TTL.USER,
  );

  if (!user) throw new UnauthorizedException();
  // ...rest of guard
}
```

2.4 Distributed Lock — Prevent Duplicate Processing

```
// src/shared/services/distributed-lock.service.ts

@Injectable()
export class DistributedLockService {
  constructor(private readonly cache: CacheService) {}

  /**
   * Acquire a lock. Returns true if acquired, false if already held.
   * Lock is auto-released after ttlSeconds (prevents deadlocks).
   */
  async acquire(
    lockKey: string,
    ttlSeconds: number = 10,
    owner: string = crypto.randomUUID(),
  ): Promise<{ acquired: boolean; owner: string }> {
    const acquired = await this.cache.setNX(lockKey, owner, ttlSeconds);
    return { acquired, owner };
  }

  /**
   * Release a lock only if we own it (prevents releasing someone else's lock).
   * Uses a Lua script for atomic check-and-delete.
   */
  async release(lockKey: string, owner: string): Promise<boolean> {
    // Lua script: check owner matches, then delete — atomic
    const script = `
      if redis.call("get", KEYS[1]) == ARGV[1] then
        return redis.call("del", KEYS[1])
      else
        return 0
      end
    `;
    const client = (this.cache as any).cacheManager.store.client;
    const result = await client.eval(script, 1, lockKey, owner);
    return result === 1;
  }

  /**
   * Run fn with a distributed lock. Throws if lock can't be acquired.
   */
  async withLock<T>(
```

```
    lockKey: string,
    fn: () => Promise<T>,
    ttlSeconds: number = 10,
  ): Promise<T> {
    const { acquired, owner } = await this.acquire(lockKey, ttlSeconds);
    if (!acquired) {
      throw new ConflictException(`Resource locked: ${lockKey}`);
    }
    try {
      return await fn();
    } finally {
      await this.release(lockKey, owner);
    }
  }
}
```

2.5 Idempotency — Fix D-1 (Duplicate Payin) and D-3 (Duplicate Webhook)

```
// src/utils/idempotency.util.ts

/**
 * Use this on ALL payin creation endpoints and webhook handlers.
 */
export class IdempotencyService {
  constructor(private readonly cache: CacheService) {}

  /**
   * Check if this payin was already processed.
   * Returns existing orderId if duplicate, null if new.
   */
  async checkPayinIdempotency(
    idempotencyKey: string,  // merchant provides this in request header
  ): Promise<string | null> {
    return this.cache.get<string>(
      REDIS_KEYS.PAYIN_IDEMPOTENCY(idempotencyKey),
    );
  }

  async recordPayinIdempotency(
    idempotencyKey: string,
    orderId: string,
  ): Promise<void> {
    await this.cache.set(
      REDIS_KEYS.PAYIN_IDEMPOTENCY(idempotencyKey),
      orderId,
      REDIS_TTL.IDEMPOTENCY_PAYIN,
    );
  }

  /**
```

```
   * Check if this webhook was already processed.
   * Returns true if duplicate (skip processing), false if new.
   */
  async isWebhookDuplicate(orderId: string, pgRef: string): Promise<boolean> {
    const key = REDIS_KEYS.WEBHOOK_IDEMPOTENCY(orderId, pgRef);
    // setNX: returns true if NEW (we just set it), false if already existed
    const isNew = await this.cache.setNX(key, "1", REDIS_TTL.IDEMPOTENCY_WEBHOOK);
    return !isNew;  // true = duplicate
  }
}
```

Usage in webhook handler:

```
// In your BasePayinWebhookService
async handleWebhook(body: any, pgRef: string, orderId: string) {
  // 1. Deduplication check — fix D-3
  const isDuplicate = await this.idempotencyService.isWebhookDuplicate(orderId,
pgRef);
  if (isDuplicate) {
    this.logger.log(`Duplicate webhook ignored: ${orderId}/${pgRef}`);
    return { received: true };  // ACK to PG, don't process again
  }

  // 2. Find order — handle D-2 race condition
  const order = await this.findOrderWithRetry(orderId, 3, 500);
  if (!order) {
    // Order doesn't exist even after retries — could be pre-D-2 race
    // Do NOT acknowledge — let PG retry later when order is created
    throw new NotFoundException(`Order not found: ${orderId}`);
  }

  // 3. Process...
}

private async findOrderWithRetry(
  orderId: string,
  maxAttempts: number,
  delayMs: number,
): Promise<PayInOrdersEntity | null> {
  for (let i = 0; i < maxAttempts; i++) {
    const order = await this.payinOrdersRepo.findOne({ where: { orderId } });
    if (order) return order;
    if (i < maxAttempts - 1) await new Promise(r => setTimeout(r, delayMs));
  }
  return null;
}
```

Part 3 — Queue Architecture: BullMQ Upgrade

3.1 Migrate from Bull to BullMQ

Bull is in maintenance mode. BullMQ is its successor — same API, better performance, TypeScript-first, supports per-job rate limiting and priority queues.

```
pnpm remove @nestjs/bull bull
pnpm add @nestjs/bullmq bullmq
```

```
// src/app.module.ts — BullMQ config
import { BullModule } from "@nestjs/bullmq";

BullModule.forRoot({
  connection: {
    host: process.env.REDIS_HOST,
    port: +process.env.REDIS_PORT,
    password: process.env.REDIS_PASSWORD,
    enableOfflineQueue: false,   // fail fast if Redis is down
    enableAutoPipelining: true,  // automatically batch commands
    maxRetriesPerRequest: 3,
  },
}),
```

3.2 Payin Queue — Fix D-2, D-4, and Add DLQ

```
// src/modules/payments/processors/payin.processor.ts

@Processor("payin-orders", {
  concurrency: 10,              // up from 5
  limiter: {
    max: 500,                   // max 500 jobs per
    duration: 1000,             // 1 second (rate limiting)
  },
})
export class PayinProcessor extends WorkerHost {
  constructor(
    @InjectDataSource() private dataSource: DataSource,
    private readonly idempotencyService: IdempotencyService,
    private readonly logger: Logger,
  ) {
    super();
  }

  async process(job: Job<PayinJobData>): Promise<void> {
    const { orderId, userId, amount, integrationCode, checkoutData } = job.data;

    const queryRunner = this.dataSource.createQueryRunner();
    await queryRunner.connect();
    await queryRunner.startTransaction("READ COMMITTED");

    try {
      // Check if already inserted (handles Bull retry after crash — fixes D-4
```

```
partial retry)
      const existing = await queryRunner.manager.findOne(PayInOrdersEntity, {
        where: { orderId },
        select: ["id"],
      });

      if (existing) {
        this.logger.warn(`Payin order already exists, skipping: ${orderId}`);
        await queryRunner.rollbackTransaction();
        return;  // idempotent — safe to skip
      }

      // Insert payin order
      const payinOrder = queryRunner.manager.create(PayInOrdersEntity, {
        orderId,
        userId,
        amount,
        status: PAYMENT_STATUS.PENDING,
        // ...rest of fields
      });
      await queryRunner.manager.save(payinOrder);

      // Insert transaction record
      // ...

      await queryRunner.commitTransaction();

    } catch (error) {
      await queryRunner.rollbackTransaction();
      this.logger.error(`Payin processor failed for ${orderId}:`, error);
      throw error;  // BullMQ will retry per job options
    } finally {
      await queryRunner.release();
    }
  }
}
```

Add Dead Letter Queue — fixes A-2:

```
// src/modules/payments/payments.module.ts
BullModule.registerQueue({
  name: "payin-orders",
  defaultJobOptions: {
    attempts: 3,
    backoff: { type: "exponential", delay: 2000 },
    removeOnComplete: 500,
    removeOnFail: false,   // KEEP failed jobs — for debugging
  },
}),

// Dead letter queue — receives jobs after all retries exhausted
```

```
BullModule.registerQueue({ name: "payin-orders-dlq" }),

// In the processor:
@OnWorkerEvent("failed")
async onFailed(job: Job, error: Error) {
  if (job.attemptsMade >= job.opts.attempts) {
    // Move to DLQ for manual review
    await this.dlqQueue.add("failed-payin", {
      originalJob: job.data,
      error: error.message,
      failedAt: new Date().toISOString(),
    }, { removeOnComplete: false, removeOnFail: false });

    // Alert the engineering team
    this.logger.error(`PAYIN JOB PERMANENTLY FAILED — moved to DLQ:
${job.data.orderId}`);
    // TODO: Send Slack/PagerDuty alert here
  }
}
```

3.3 Webhook Queue — Fix D-3 and Add Rate Control

Instead of processing webhooks synchronously in the HTTP handler, queue them:

```
// src/modules/payments/webhook.controller.ts

@Post("payin/webhook")
@UseGuards(WebhookGuard)
async handlePayinWebhook(@Body() body: any) {
  // Enqueue immediately — return 200 to PG fast
  await this.webhookQueue.add("process-webhook", {
    body,
    receivedAt: Date.now(),
    source: "NXT",
  }, {
    // Deduplication by PG's reference number
    jobId: `webhook:nxt:${body.reference_id}`,  // BullMQ deduplicates by jobId!
    attempts: 5,
    backoff: { type: "exponential", delay: 1000 },
  });

  return { received: true };  // Always return 200 to PG
}
```

Using `jobId` for deduplication is the cleanest pattern — BullMQ rejects adding a job with the same `jobId` if one already exists in the queue. This gives you free webhook deduplication without a Redis lookup.

Part 4 — Senior Developer Code Patterns

4.1 Repository Pattern — Separate DB from Business Logic

Never call TypeORM directly in services. Use typed repositories.

```
// src/modules/payments/repositories/payin-orders.repository.ts

@Injectable()
export class PayinOrdersRepository {
  constructor(
    @InjectRepository(PayInOrdersEntity)
    private readonly repo: Repository<PayInOrdersEntity>,
    @InjectDataSource()
    private readonly dataSource: DataSource,
  ) {}

  async findByOrderId(orderId: string): Promise<PayInOrdersEntity | null> {
    return this.repo.findOne({ where: { orderId } });
  }

  async findByUserIdPaginated(
    userId: string,
    page: number,
    limit: number,
    filters: PayinListFilters,
  ): Promise<[PayInOrdersEntity[], number]> {
    const qb = this.repo.createQueryBuilder("po")
      .where("po.user_id = :userId", { userId })
      .orderBy("po.created_at", "DESC")
      .skip((page - 1) * limit)
      .take(limit);

    if (filters.status) {
      qb.andWhere("po.status = :status", { status: filters.status });
    }
    if (filters.startDate) {
      qb.andWhere("po.created_at >= :startDate", { startDate: filters.startDate });
    }
    if (filters.endDate) {
      qb.andWhere("po.created_at <= :endDate", { endDate: filters.endDate });
    }

    return qb.getManyAndCount();
  }

  async updateStatusWithUtr(
    orderId: string,
    status: PAYMENT_STATUS,
    utr: string,
    queryRunner: QueryRunner,
  ): Promise<void> {
    await queryRunner.manager.update(PayInOrdersEntity,
      { orderId },
```

```
      { status, utr, updatedAt: new Date() },
    );
  }

  /**
   * Bulk stats query — runs on read replica (analytics pattern)
   * Uses raw SQL for performance (TypeORM ORM overhead is too high for
aggregations)
   */
  async getDailyStats(userId: string, date: string): Promise<DailyStats> {
    const result = await this.dataSource.query(`
      SELECT
        COUNT(*) FILTER (WHERE status = 'SUCCESS') AS success_count,
        COUNT(*) FILTER (WHERE status = 'FAILED') AS failed_count,
        COUNT(*) FILTER (WHERE status = 'PENDING') AS pending_count,
        COALESCE(SUM(amount) FILTER (WHERE status = 'SUCCESS'), 0) AS
success_volume,
        COALESCE(AVG(amount) FILTER (WHERE status = 'SUCCESS'), 0) AS
avg_ticket_size
      FROM payin_orders
      WHERE user_id = $1
        AND DATE(created_at AT TIME ZONE 'Asia/Kolkata') = $2::date
    `, [userId, date]);

    return result[0];
  }
}
```

4.2 Circuit Breaker — For External PG Calls

When a PG is down, fail fast instead of hanging 6 PM2 instances with waiting requests.

```
// src/shared/circuit-breaker/circuit-breaker.service.ts

export enum CircuitState { CLOSED, OPEN, HALF_OPEN }

interface CircuitBreakerOptions {
  failureThreshold: number;   // # failures before opening
  successThreshold: number;   // # successes in HALF_OPEN before closing
  timeout: number;            // ms to wait before trying HALF_OPEN
  requestTimeout: number;     // ms before treating a request as failed
}

@Injectable()
export class CircuitBreakerService {
  private circuits = new Map<string, {
    state: CircuitState;
    failureCount: number;
    successCount: number;
    nextAttempt: number;
  }>();
```

```
  async execute<T>(
    circuitName: string,
    fn: () => Promise<T>,
    options: CircuitBreakerOptions = {
      failureThreshold: 5,
      successThreshold: 2,
      timeout: 30000,       // 30s open before half-open
      requestTimeout: 5000, // 5s per request
    },
  ): Promise<T> {
    const circuit = this.circuits.get(circuitName) ?? {
      state: CircuitState.CLOSED,
      failureCount: 0,
      successCount: 0,
      nextAttempt: 0,
    };

    if (circuit.state === CircuitState.OPEN) {
      if (Date.now() < circuit.nextAttempt) {
        throw new ServiceUnavailableException(
          `Payment gateway ${circuitName} is currently unavailable. Please try again
shortly.`
        );
      }
      circuit.state = CircuitState.HALF_OPEN;
      circuit.successCount = 0;
    }

    try {
      // Race request against timeout
      const result = await Promise.race([
        fn(),
        new Promise<never>((_, reject) =>
          setTimeout(() => reject(new Error("Request timeout")),
options.requestTimeout)
        ),
      ]);

      // Success
      if (circuit.state === CircuitState.HALF_OPEN) {
        circuit.successCount++;
        if (circuit.successCount >= options.successThreshold) {
          circuit.state = CircuitState.CLOSED;
          circuit.failureCount = 0;
        }
      }
      this.circuits.set(circuitName, circuit);
      return result;

    } catch (error) {
      circuit.failureCount++;
```

```
      if (circuit.failureCount >= options.failureThreshold) {
        circuit.state = CircuitState.OPEN;
        circuit.nextAttempt = Date.now() + options.timeout;
        // Alert: this PG is now circuit-breaker OPEN
      }
      this.circuits.set(circuitName, circuit);
      throw error;
    }
  }

  getStatus(circuitName: string): CircuitState {
    return this.circuits.get(circuitName)?.state ?? CircuitState.CLOSED;
  }
}

// Usage in OnikPayinService:
async createPayin(dto: CreatePayinDto, user: UsersEntity) {
  return this.circuitBreaker.execute("ONIK", async () => {
    return this.axiosService.post(onikEndpoint, payload);
  });
}
```

4.3 Rate Limiter — Fix S-2

```
// Install: pnpm add @nestjs/throttler

// src/app.module.ts
ThrottlerModule.forRoot([
  {
    name: "short",
    ttl: 1000,    // 1 second
    limit: 20,    // 20 requests per second per IP
  },
  {
    name: "medium",
    ttl: 60000,   // 1 minute
    limit: 200,   // 200 requests per minute per IP
  },
  {
    name: "long",
    ttl: 3600000, // 1 hour
    limit: 1000,  // 1000 requests per hour per IP
  },
]),

// In AppModule providers:
{
  provide: APP_GUARD,
  useClass: ThrottlerGuard,
},
```

```
// Override limits per endpoint:
@Throttle({ short: { limit: 3, ttl: 60000 } })  // 3 OTPs per minute
@Post("send-signup-otp")
async sendSignupOtp(@Body() dto: SendOtpDto) { ... }

@Throttle({ short: { limit: 5, ttl: 300000 } }) // 5 attempts per 5 minutes
@Post("login")
async login(@Body() dto: LoginDto) { ... }

// Skip throttling on webhook endpoints (PGs may send many fast)
@SkipThrottle()
@Post("payin/webhook")
async handleWebhook() { ... }
```

4.4 Correlation IDs — Fix A-9

```
// src/middleware/correlation-id.middleware.ts
import { v4 as uuidv4 } from "uuid";
import { AsyncLocalStorage } from "async_hooks";

// Global store — works across async boundaries
export const correlationStore = new AsyncLocalStorage<{ traceId: string }>();

@Injectable()
export class CorrelationIdMiddleware implements NestMiddleware {
  use(req: Request, res: Response, next: NextFunction) {
    const traceId = (req.headers["x-trace-id"] as string) ?? `rf-${uuidv4()}`;
    res.setHeader("x-trace-id", traceId);

    correlationStore.run({ traceId }, () => next());
  }
}

// src/logger/logger.service.ts — add traceId to every log
export class AppLogger extends Logger {
  log(message: string, context?: string) {
    const store = correlationStore.getStore();
    const traceId = store?.traceId ?? "no-trace";
    super.log(`[${traceId}] ${message}`, context);
  }
  error(message: string, trace?: string, context?: string) {
    const store = correlationStore.getStore();
    const traceId = store?.traceId ?? "no-trace";
    super.error(`[${traceId}] ${message}`, trace, context);
  }
}

// Register in main.ts:
app.use(new CorrelationIdMiddleware().use);
```

4.5 Graceful Shutdown — Fix A-8

```
// src/main.ts
async function bootstrap() {
  const app = await NestFactory.create(AppModule);

  // Enable graceful shutdown — SIGTERM triggers onModuleDestroy()
  app.enableShutdownHooks();

  // ...rest of config

  await app.listen(port);
}

// In your Bull processors:
@Injectable()
export class PayinProcessor extends WorkerHost implements OnModuleDestroy {
  async onModuleDestroy() {
    // Wait for in-flight jobs to complete before shutdown
    await this.worker.close();
  }
}

// In PaymentsModule:
@Module({})
export class PaymentsModule implements OnModuleDestroy {
  constructor(
    @InjectQueue("payin-orders") private payinQueue: Queue,
  ) {}

  async onModuleDestroy() {
    // Pause queue — stops picking new jobs
    await this.payinQueue.pause();
  }
}
```

4.6 Health Checks — Fix DEV-1

```
// Install: pnpm add @nestjs/terminus

// src/modules/health/health.module.ts
@Module({
  imports: [TerminusModule, TypeOrmModule, HttpModule],
  controllers: [HealthController],
})
export class HealthModule {}

// src/modules/health/health.controller.ts
@Controller("health")
```

```
export class HealthController {
  constructor(
    private health: HealthCheckService,
    private db: TypeOrmHealthIndicator,
    private redis: MicroserviceHealthIndicator,
    private memory: MemoryHealthIndicator,
    private disk: DiskHealthIndicator,
  ) {}

  @Get()
  @Public()
  @HealthCheck()
  async check() {
    return this.health.check([
      // PostgreSQL connectivity
      () => this.db.pingCheck("database", { timeout: 3000 }),

      // Memory — warn if heap > 400MB
      () => this.memory.checkHeap("memory_heap", 400 * 1024 * 1024),

      // Redis
      () => this.checkRedis(),
    ]);
  }

  @Get("ready")
  @Public()
  async readiness() {
    // Called by K8s/ECS before routing traffic
    // More strict — all dependencies must be ready
    return this.health.check([
      () => this.db.pingCheck("database"),
      () => this.checkRedis(),
      () => this.checkQueueDepth(),
    ]);
  }

  private async checkRedis() {
    // Custom check via your CacheService
    // ...
  }

  private async checkQueueDepth() {
    // Alert if payin queue depth > 10,000 (processing falling behind)
    // ...
  }
}
```

Part 5 — The Complete Architecture for 3Cr/day

```
                    ┌──────────────────────────────────────┐
                    │           INGRESS LAYER              │
                    │  Nginx / AWS ALB (rate limiting + TLS)  │
                    └──────────────────────────────────────┘
                                   │
                                   │ L7 load balance
                ┌──────────────────┼──────────────────────┐
                │                  │                      │
        ┌───────▼────────┐  ┌──────▼─────────┐  ┌──────────▼──────┐
        │  NestJS pod 1  │  │ NestJS pod 2   │  │  NestJS pod N   │
        │  (payin/auth)  │  │(analytics/KYC)│  │  (payout/admin) │
        └────────────────┘  └────────────────┘  └─────────────────┘
                │                  │                      │
                └──────────────────┼──────────────────────┘
                                   │
                                   │
            ┌──────────────────────┼──────────────────────┐
            │                      │                      │
    ┌───────▼────────┐     ┌────────▼──────────┐  ┌────────▼────────┐
    │  PgBouncer     │     │   Redis Cluster   │  │   BullMQ        │
    │  (6432)        │     │  3 master + 3 replica│  │  Queue Workers  │
    └────────────────┘     └────────────────────┘  └─────────────────┘
            │                      │                      │
    ┌───────▼──────────────────────▼──────────────────────▼──┐
    │                    STORAGE LAYER                        │
    │                                                         │
    │  PostgreSQL Primary          PostgreSQL Replica 1       │
    │  (writes only)               (app reads)                │
    │                                                         │
    │  PostgreSQL Replica 2        S3 (KYC docs)              │
    │  (analytics only)                                       │
    └─────────────────────────────────────────────────────────┘
```

Part 6 — Step-by-Step Implementation Order

Do these in order. Each step unblocks the next.

Sprint 1 — Stop the Bleeding (Week 1–2)

> *You're at 45 req/sec with 64% success. These fixes will get you to 200+ req/sec.*

1. **Install PgBouncer** and update `db.config.ts` to point at it (1 day)
2. **Configure DB pool**: `max: 25, min: 5, prepareThreshold: 0` (already 0 ✓) (2 hours)
3. **Add `@nestjs/throttler`** to all public endpoints (1 day)
4. **Fix webhook IP guard** to use `req.socket.remoteAddress` correctly (2 hours)
5. **Add idempotency key check** on payin endpoints (1 day)
6. **Re-enable KYC guards** — uncomment in `AppModule` (1 hour)

Sprint 2 — Reliability (Week 3–4)

> *Get to 500 req/sec with 99%+ success rate.*

7. **Add webhook deduplication** via BullMQ `jobId` (1 day)
8. **Implement `findOrderWithRetry`** in webhook handlers (1 day)

9. **Migrate Bull → BullMQ** (2 days)
10. **Add DLQ** for failed payin/payout jobs (1 day)
11. **Add health check endpoint** with Redis + DB checks (1 day)
12. **Fix wallet update** with `pg_advisory_xact_lock` (2 days)

Sprint 3 — Database Scaling (Week 5–6)

> *Get to 1,000 TPS with read replicas.*

13. **Add PostgreSQL read replica** and configure TypeORM replication (2 days)
14. **Partition `payin_orders` and `transactions`** tables (3 days, requires maintenance window)
15. **Add missing indexes** via `CONCURRENTLY` migrations (1 day)
16. **Run `pg_stat_statements`** analysis to find and fix top 10 slow queries (2 days)

Sprint 4 — Redis Hardening (Week 7–8)

> *Eliminate Redis as SPOF.*

17. **Deploy Redis Cluster** (3 master + 3 replica) (2 days)
18. **Rewrite Redis key constants** using new `REDIS_KEYS` constants (1 day)
19. **Add distributed lock service** for wallet updates (1 day)
20. **Fix API key cache invalidation** on key deletion (1 day)

Sprint 5 — Orchestration Ready (Week 9–12)

> *Scale beyond current PG limitations.*

21. **Add circuit breakers** on all external PG calls (3 days)
22. **Build PG smart router** with failover (5 days)
23. **Add correlation IDs** and structured logging (2 days)
24. **PII encryption** at rest for bank/PAN/Aadhaar fields (3 days)
25. **Audit log table** for all admin actions (2 days)
26. **Add reconciliation cron job** (5 days)

Part 7 — Performance Benchmarks to Hit

After each sprint, run a load test (`k6` or `autocannon`) and verify:

```
Sprint 1 complete:  200 req/sec, 99% success, p99 latency < 500ms
Sprint 2 complete:  500 req/sec, 99.5% success, zero duplicate webhooks
Sprint 3 complete:  1,000 req/sec, 99.9% success, read queries on replica
Sprint 4 complete:  1,000 req/sec + Redis failover surviving a node crash
Sprint 5 complete:  1,000 TPS × 86,400s = 86.4M transactions/day capacity
                    (3 Crore = 30M/day = comfortably within capacity at ~35% load)
```

3 Crore transactions/day at 35% peak load = your system is safely over-provisioned.

Quick Reference Cheatsheet

```
DB RULES:
  ✓ All writes  → PgBouncer → PostgreSQL Primary
  ✓ App reads   → PgBouncer → PostgreSQL Replica 1
```

```
  ✓ Analytics   → Direct    → PostgreSQL Replica 2
  ✓ Aggregations use raw SQL, not ORM .find()
  ✓ All financial updates inside QueryRunner + pg_advisory_xact_lock
  ✓ Every new table created with monthly PARTITION BY RANGE(created_at)
  ✓ prepareThreshold: 0 always (PgBouncer transaction mode)

REDIS RULES:
  ✓ Every key has a TTL — no exceptions
  ✓ Key format: {namespace}:{entity}:{id}
  ✓ Financial atomicity → pg_advisory_xact_lock (not Redis locks)
  ✓ Webhook dedup → BullMQ jobId (not Redis SETNX)
  ✓ Idempotency keys → Redis SETNX with 24h TTL
  ✓ Never store PII in Redis (no bank account numbers, no PAN)

QUEUE RULES:
  ✓ Webhook handlers always return 200 to PG immediately — queue first, process later
  ✓ Every queue has a DLQ + alerting on permanent failure
  ✓ Every processor is idempotent — safe to run twice with same input
  ✓ Graceful shutdown: pause queue → finish in-flight → close worker

CODE RULES:
  ✓ No TypeORM ORM calls in analytics — raw SQL only
  ✓ Repository pattern — no direct repo injection in controllers
  ✓ Every external PG call wrapped in circuit breaker
  ✓ Every public endpoint has @Throttle() applied
  ✓ Every request has a traceId threaded through all logs
```

End of Document — Rupeeflow Scale Playbook — March 2026 Classification: Internal Engineering — Confidential